Statistical Inference and Modeling

2102575

Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2025

Lectures

- Lecture 1: Introduction
- Lecture 2: Model selection
- Lecture 3: Resampling
- Lecture 4: Linear regression
- Lecture 5: Robust regression
- Lecture 6: Classification
- Lecture 7: Logistic regression
- Lecture 8: Neural networks
- Lecture 9: K-nearest neighbor
- Lecture 10: Bayesian decision, LDA, QDA, and naive Bayes
- Lecture 11: Support vector machine
- Lecture 12: Tree-based methods
- Lecture 13: Bagging, random forecast, boosting
- Lecture 14: Principal component analysis
- Lecture 15: Clustering algorithms
- Lecture 16: Gaussian mixture model clustering
- Lecture 17: DBSCAN clustering

13. Tree-based methods

2102575

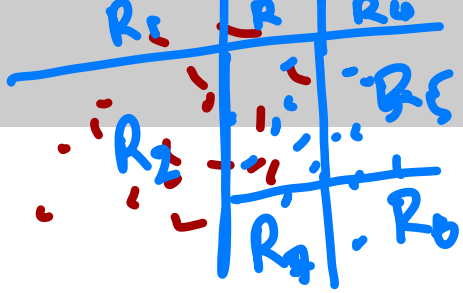
Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2024

2 Decision tree learning
Decision tree learning

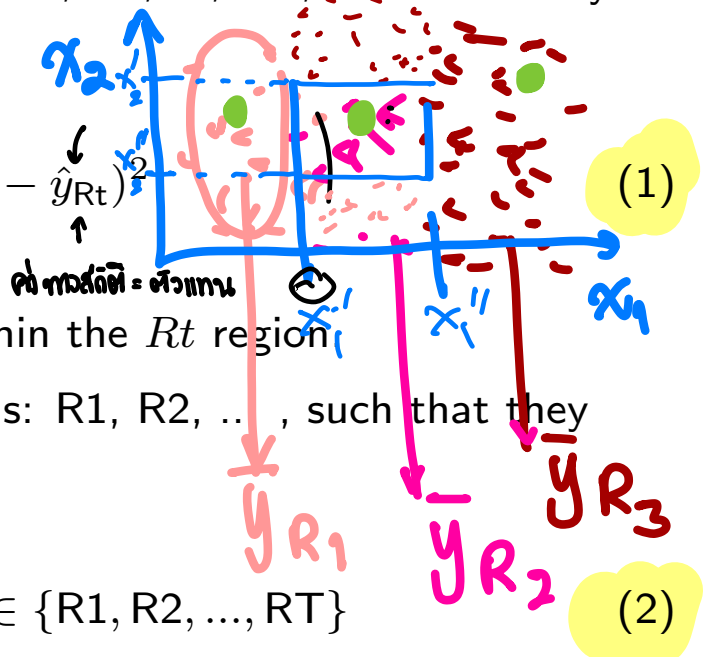
Decision tree — learning formulation



Let a set of training samples be denoted with $\{\boldsymbol{x}_j, y_j\}_{j=1}^{m}$. For every observation that falls into the region (*e.g.*, R1, R2, ...), we make the same prediction as the training samples in the region.

Regression. Our learning goal is to construct the regions: R1, R2, ..., RT, such that they minimize the residual sum of squares (RSS):

$$\underset{R_1, R_2, \ldots R_T}{\text{minimize}} \sum_{\text{Rt} \in \{\text{R1, R2, ...}\}} \sum_{j \in \text{Rt}} (y_j - \hat{y}_{\text{Rt}})^2 \qquad (1)$$



where $\hat{y}_{Rt}$ is the sample mean of the training samples within the Rt region.

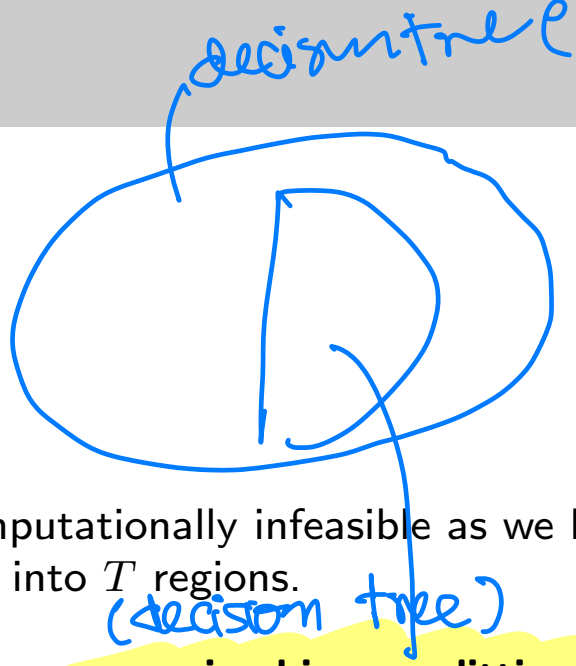
Classification. Our learning goal is to construct the regions: R1, R2, .. , such that they minimize **the classification error rate**:

$$\underset{R_1, R_2, \ldots R_T}{\text{minimize}} \{1 - \max_{c \in C} \hat{p}(y_{\text{Rt}} = c)\}, \quad \forall\ \text{Rt} \in \{\text{R1}, \text{R2}, \ldots, \text{RT}\} \qquad (2)$$

Here $\hat{p}(y_{\text{Rt}} = c)$:= the probability that a training sample in the region Rt is in the cth class.

$$\frac{\sum (y_j \in C) \,\&\, (y_j \in R_t)}{\sum (y_j \in R_t)}$$

From learning formulation to building a tree.

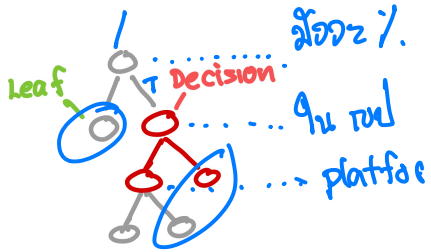


Note:

- Unfortunately, solving both Eq.(1) and Eq.(2) is computationally infeasible as we have to consider every possible partition of the feature space into T regions.
- To solve this, **a top-down, greedy approach** known as **a recursive binary splitting** is used.
- *top-down*: the alg. begins at the top of the tree (at which point binary all observations belong to a single region) and then successively splits the splitting predictor space.
- *greedy*: because at each step of the tree-building process, the best split is made at that particular step (not a globally optimal).

Decision tree — the structure

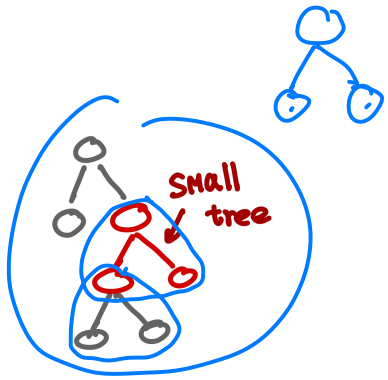
* Binary splitting → Binary classification.



- A (binary) tree is a graph representation for how one will split data samples into several regions based on a set of threshold values.
- If a node has children nodes, it is a decision node.

- A decision node contains
  - a threshold value
  - A decision node is a root node for a (small, binary) tree.
  - Therefore, it will associate with its own children nodes, *i.e.*, left (L) and right (R) children nodes.
  - Its child nodes can be either a decision node or leaf nodes.
- A leaf node contains
  - value for the classification/regression result. *
  - A leaf node is a root node that does not connect to any subsequent (small, binary) trees.

node ⟷ region.



- One big (binary) tree can contain multiple small (binary) trees.
- A (small, binary) tree consists of two children: left and right children.

Decision tree learning—what to configure/learn

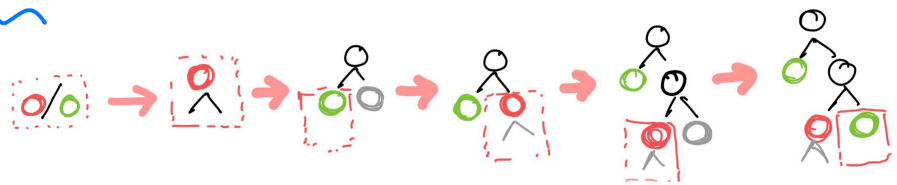
- Tasks (fixed operation):
  - Classification:

$$\hat{y} = \text{Majority}\{y_j\}_{j \in \text{Leaf}}$$

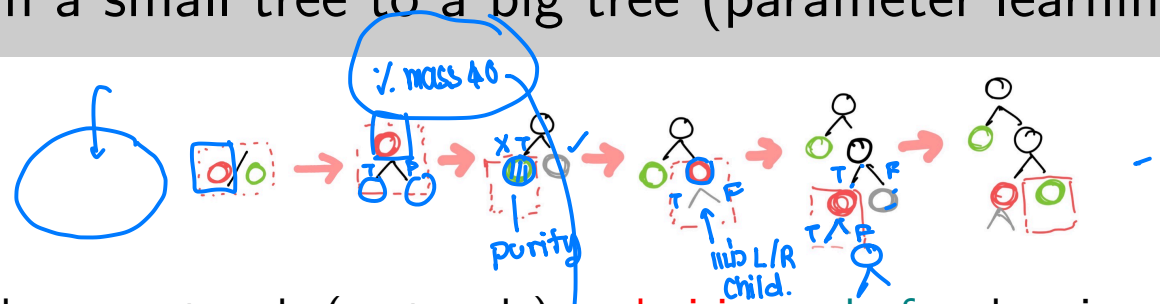
  - Regression:

$$\hat{y} = \frac{1}{|\text{Leaf}|} \sum_{i \in \text{Leaf}} y_i$$

    where Leaf is the index set of **training samples** that fall into an output region after threshold operation (*e.g.*, Leaf can be either R1, R2, or R3).
- Tree structure:
  - Node splittings (how is a decision node is going to be split?, *e.g.*, binary splitting) → Hyper-parameters.
  - The diameter/width of a tree → Hyper-parameters.
  - Splitting threshold → Parameters.



Building from a small tree to a big tree (parameter learning)



1. Assigning the current node (root node) as decision or leaf node using all the data samples. The assignment is done by checking **checking whether the samples in this node is pure (can you split these data samples)**?

Yes The root node is assigned as decision node.
Find **the best splitting threshold** using the data samples.
Collect the splitting threshold.
Assign data samples to the left (L) and right (R) child nodes.
Build the tree for the L child node (recursion).
Build the tree for the R child node (recursion).

No The root node is assigned as a leaf node.
The leaf node will contain the set of samples to compute the output:
Classification:

$$\hat{y} = \text{Majority}\{y_j\}_{j \in \text{Leaf}}$$

Regression:

$$\hat{y} = \frac{1}{|\text{Leaf}|} \sum_{i \in \text{Leaf}} y_i$$

Regression — node purity

- When a node is a decision node, we have to find the threshold value associated with it.
- The threshold value should be made such that it can split the data meaningfully.

In Regression, we use sample variance of the measurements to check the node purity. Let say if a node is associated with M training samples. The node purity is

$$\frac{1}{M-1}\sum_{i=1}^{M}(y_i-\bar{y})^2$$

where $\bar{y}$ is the sample mean of training samples associated with a node.

Note:

- The higher the variance the lower the node purity.
- However, in classification, we use Entropy or Gini index to measure **node purity**. These two quantities will be discussed in next slide.
- Similar to the variance, the higher the Entropy (or Gini index) the lower the node purity.

Classification — node purity

- Entropy measures the purity via the certainty/uncertainty of the class that have been assigned.

$$-\sum_{c \in C} \hat{p}(y_{\text{Rt}} = c) \log \hat{p}(y_{\text{Rt}} = c)$$

The entropy will take on a value near zero if the $\hat{p}(y_{\text{Rt}} = c)$ are all near zero or near one.

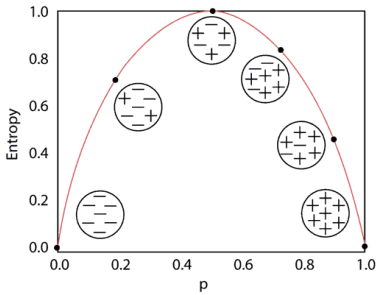
- Gini index measures the purity via the total probability of the C classes that have been assigned.

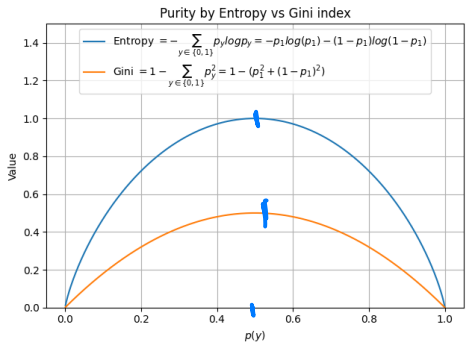
$$1 - \sum_{c \in C} \hat{p}(y_{\text{Rt}} = c)^2$$

if all of the $\hat{p}(y_{\text{Rt}} = c)$ are close to zero or one, Gini index tales a small value.

* For Entropy and Gini index, the class probability can be computed using the relative frequencies:

$$\text{Relative frequency} = \frac{\text{The number of samples in a class}}{\text{total samples}}$$





The picture is from https://blog.quantinsti.com/gini-index/.

Finding splitting thresholds (regression)

Regression: The better splitting threshold gives the more variance reduction.

$$\text{VR} = \text{Var}(\text{Parents}) - \sum_{i \in L,R} w_i \text{Var}(\text{Child}_i)$$

where w_i is the ratio of the samples in the left/right child nodes against the total number of samples without splitting. Here, $\text{Var}(\cdot)$ is the sample variance of a node.

- Variance parents Var(Parents):

$$\text{Var}(\text{Parents}) = \frac{1}{M} \sum_{i=1}^{M} (y_i - \bar{y})^2$$

- Variance of a child $\text{Var}(\text{Child}_i)$:

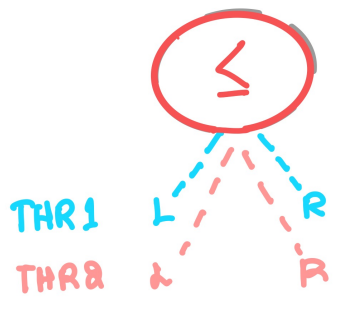
$$\text{Var}(\text{Child}_{i=L}) = \frac{1}{M_L} \sum_{i=1}^{M_L} (y_i - \bar{y}_L)^2 \qquad \text{Var}(\text{Child}_{i=R}) = \frac{1}{M_R} \sum_{i=1}^{M_R} (y_i - \bar{y}_R)^2$$

  where $\bar{y}$, $\bar{y}_L$, $\bar{y}_R$ are the sample mean before splitting, the sample mean of samples in left child nodes, and the sample mean of samples in right child nodes.
- Note: We will then try to find the threshold that gives higher VR.
- When the VR is higher, the splitting gives the more pure chi' ' '

Example:

$\text{VR}_{\text{thr1}} = 0.45 - \frac{14}{20} \times 0.35 - \frac{6}{20} \times 0.55 = 0.04$

$\text{VR}_{\text{thr2}} = 0.45 - \frac{6}{20} \times 0.1 - \frac{14}{20} \times 0.4 = 0.14 \hookleftarrow$ Better!.



Finding splitting thresholds (classification)

Classification: The better splitting threshold gives the higher information gain.

- The information gain (IG) based on entropy is defined by

$$\text{IG} = \text{Entropy(Parents)} - \sum_{i \in L,R} w_i \text{Entropy(Child}_i\text{)}$$

  where the entropy or Entropy $:= \sum_{c \in \text{Classes}} -p_c \log p_c$.

- The information gain based on Gini index (GI) is defined by

$$\text{IG} = \text{Gini(Parents)} - \sum_{i \in L,R} w_i \text{Gini(Child}_i\text{)}$$

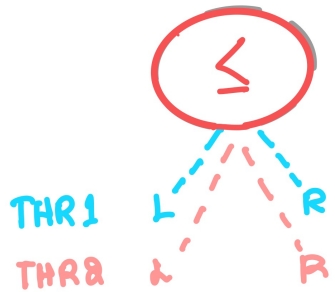
  where Gini index or Gini $:= (1 - \sum_{c \in \text{Classes}} p_c^2)$.

- Note: We will then try to find the threshold that gives higher IG.
- If IG is higher, the splitting results in the more pure child node.

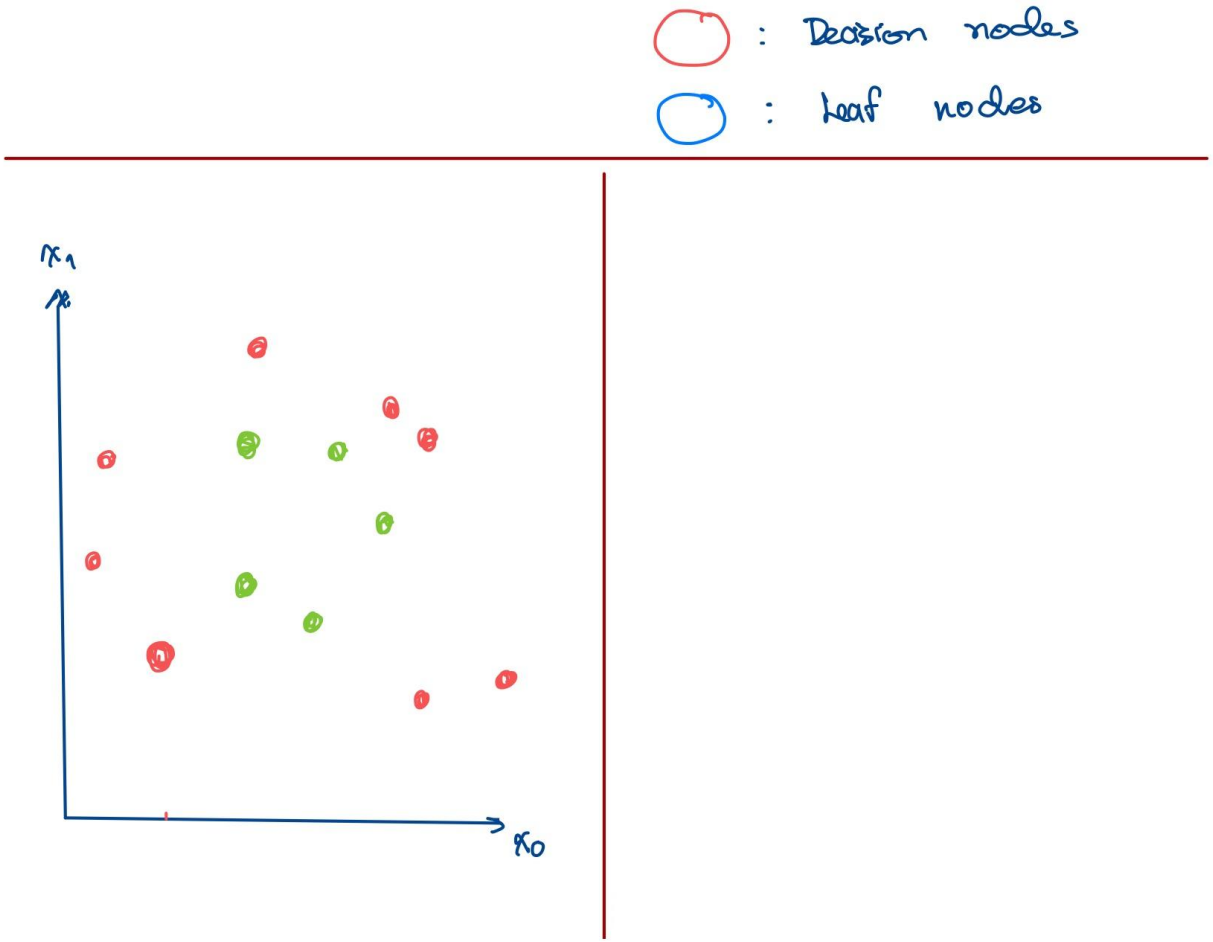
Example:

$\text{IG}_{\text{thr1}} = 0.5 - \frac{14}{20} \times 0.49 + \frac{6}{20} \times 0.44 = 0.024$

$\text{IG}_{\text{thr2}} = 0.5 - \frac{4}{20} \times 0 - \frac{16}{20} \times 0.47 = 0.129 \leftarrow$ Better!.



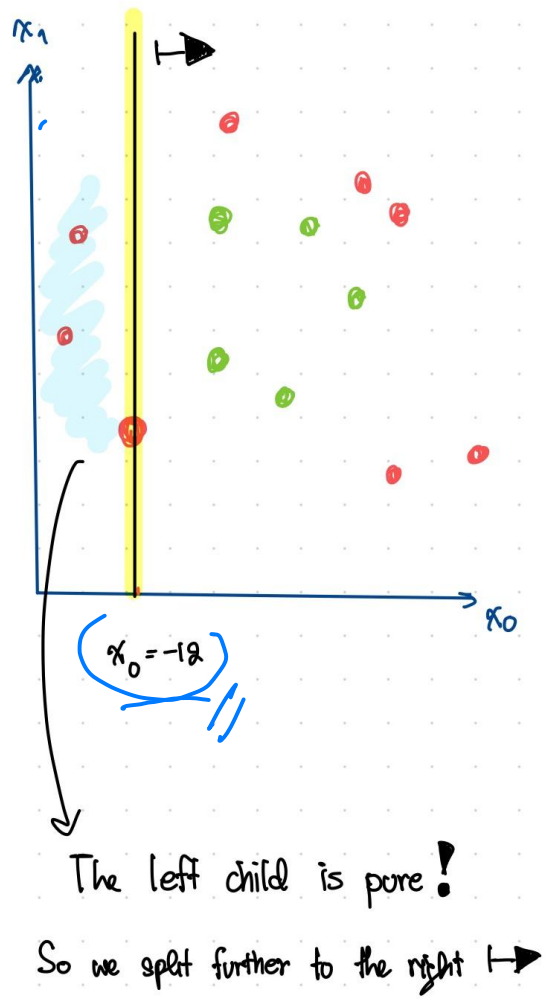
We adopted the example from https://www.youtube.com/watch?v=ZVR2Way4nwQ

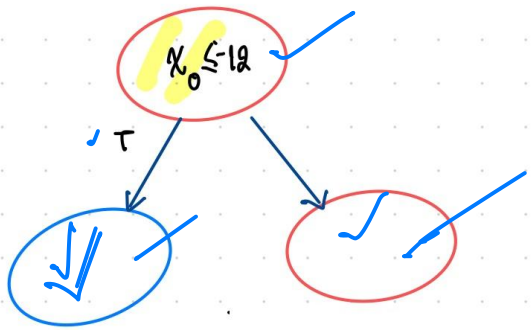


Decision tree

: Decision nodes

: Leaf nodes

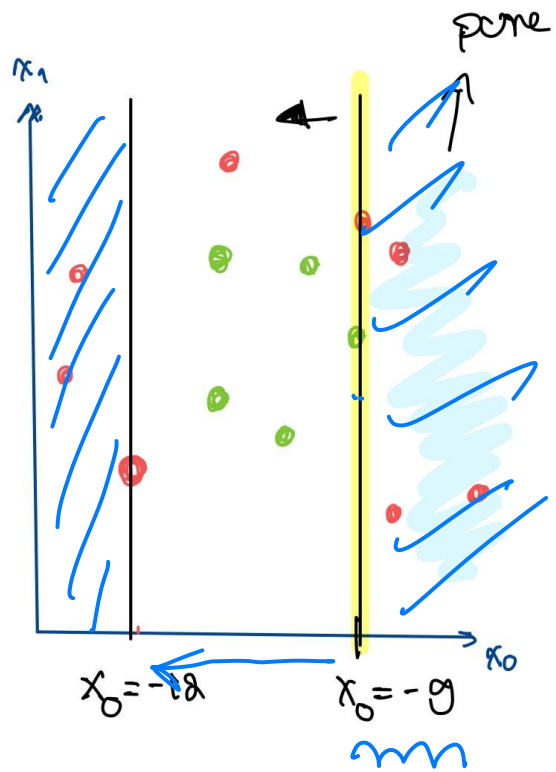




Decision tree

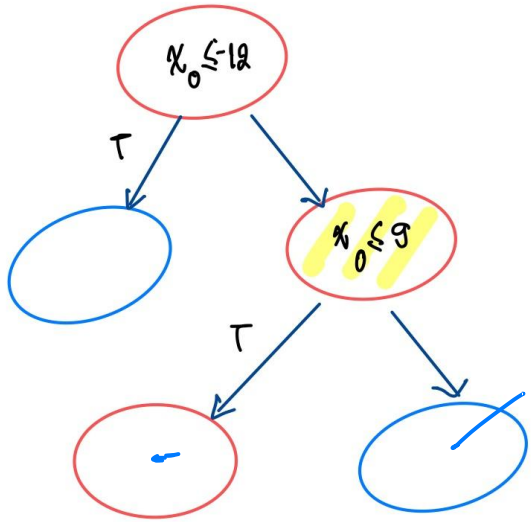
: Decision nodes

: Leaf nodes



This time, however, the right child is pure !

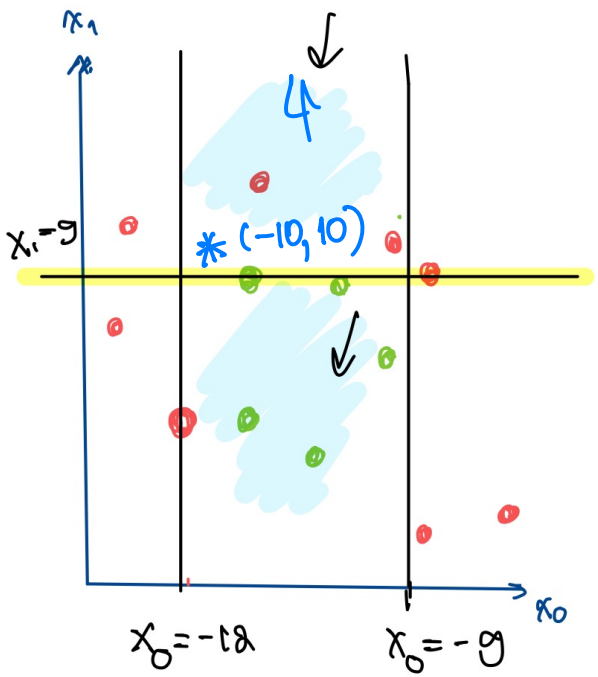
So, we will split further to the left ←

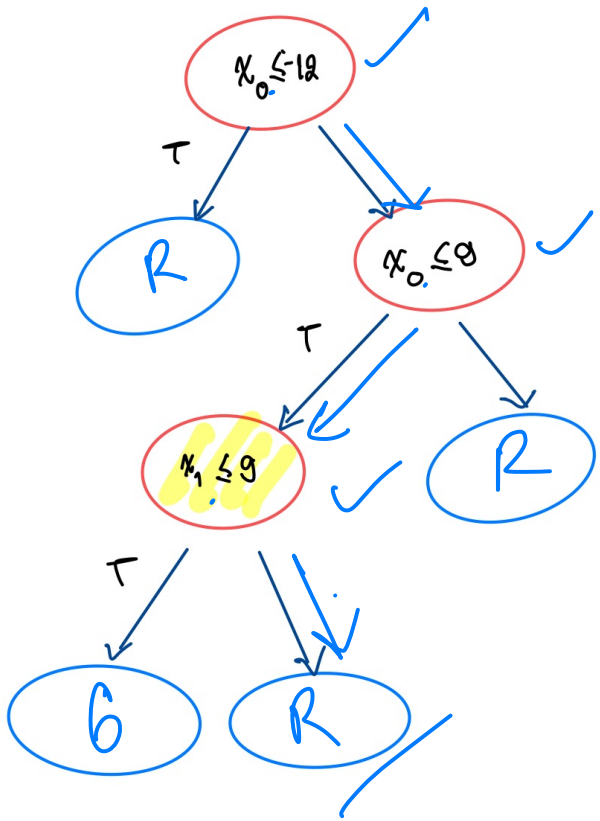


Decision tree

: Decision nodes

: Leaf nodes

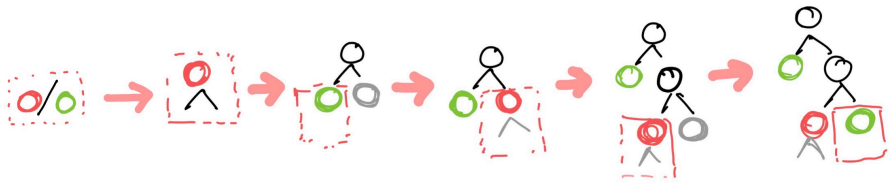




This time both sides of the childs are pure !

So, the splitting stops !

Building from a small tree to a big tree (training)



1. Assigning the current node (root node) as decision or leaf node using all the data samples.
2. Can you can split these data samples (check the criterion).

Yes The root node is assigned as decision node.

Find **the best splitting threshold** $\rightarrow$ to separate **the pure child!**

Collect the splitting threshold.

Assign data samples to the left (L) and right (R) child nodes.

Build the tree for the L child node (recursion).

Build the tree for the R child node (recursion).

No The root node is assigned as a leaf node.

The leaf node will contain the set of samples to compute the output:

Classification:

$$\hat{y} = \text{Majority}\{y_j\}_{j \in \text{Leaf}}$$

Regression:

$$\hat{y} = \frac{1}{|\text{Leaf}|} \sum_{i \in \text{Leaf}} y_i$$

Implementation

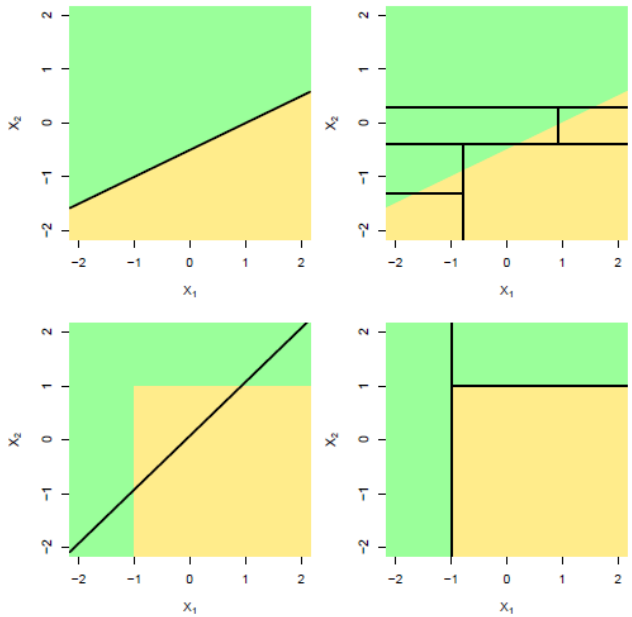
- Decision tree implementation: https://github.com/GenAI-CUEE/Statistical-Learning-EE575/tree/master/decision_tree_tutorial
- The decision tree codes used in this tutorial are adopted from Normalized Nerd: https://www.youtube.com/watch?v=sgQAhG5Q7iY.

Advantages/disadvantages

+ Trees can be displayed graphically, and are easily interpreted even by a non-expert (especially if they are small).

+ Trees can easily handle qualitative predictors without the need to create dummy variables.

– The tree-based methods can be very unstable: a small change in **training data** can result in a different tree due to the hierarchical nature where an error that occures at a high level node can propagate all the way to the leaf nodes below.
  - A tree with the longer width → higher variance and lower bias → overfitting training data.
  - A tree with the shorter width → lower variance and higher bias → underfitting training data.
  - Trees generally give higher variance than a linear regression, as shown in the figure below:



References I

[1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY, USA: Springer New York Inc., 2001.

[2] Gareth James et al. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. URL: https://hastie.su.domains/ISLP/ISLP_website.pdf.download.html.

[3] Jitkomut Songsiri. "Statistical Learning". In: in EE575 Random Processes for EE. Chulalongkorn University, 2022. Chap. 9. Support vector machine. URL: http://jitkomut.eng.chula.ac.th/ee575.html.

References I

[1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY, USA: Springer New York Inc., 2001.

[2] Gareth James et al. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. URL: https://hastie.su.domains/ISLP/ISLP_website.pdf.download.html.

[3] Jitkomut Songsiri. "Statistical Learning". In: in EE575 Random Processes for EE. Chulalongkorn University, 2022. Chap. 9. Support vector machine. URL: http://jitkomut.eng.chula.ac.th/ee575.html.